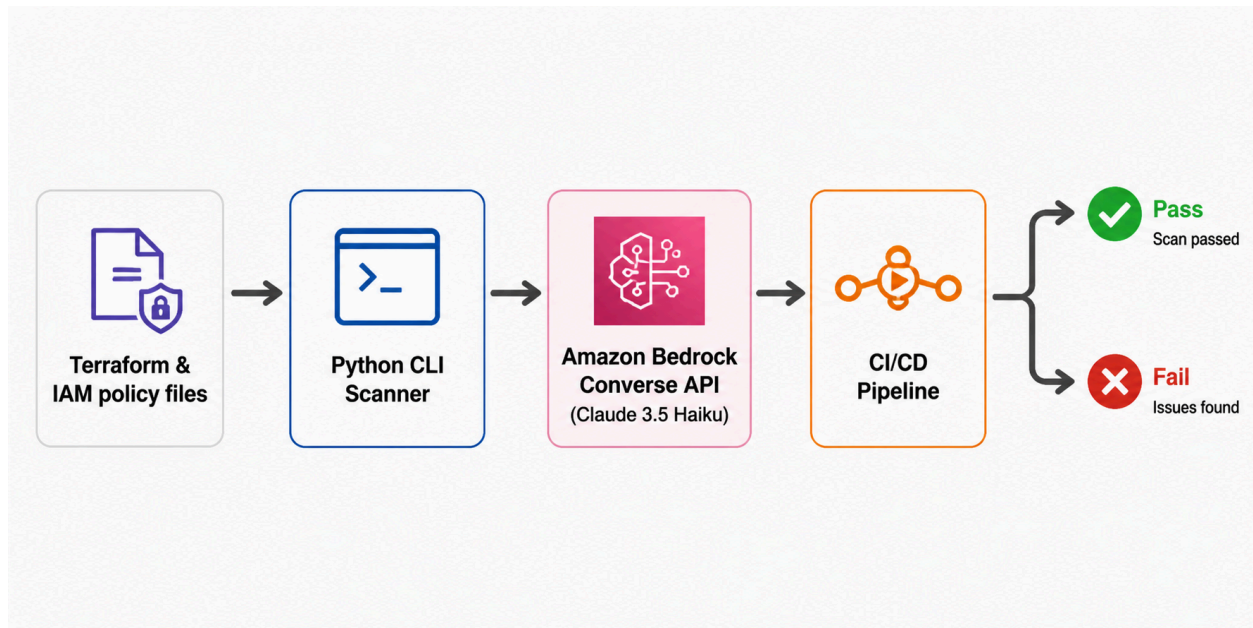


AI Python Security Scanner with CI/CD Integration



In this project, you will build a Python command-line tool that scans Terraform and IAM policy files for security vulnerabilities. It uses AI to explain each risk and suggest specific fixes. This features:

- A working **Python CLI tool** that scans Terraform `.tf` files and IAM policy JSON for common security anti-patterns.
- **AI-generated security reports** with severity ratings, plain-English risk explanations, and remediation code powered by Amazon Bedrock.
- The **AWS Bedrock Converse API** for sending structured prompts to Claude 3.5 Haiku and parsing responses
- Generate security reports that combine automated scanning with AI-powered analysis, complete with severity ratings and actionable fixes.
- Added **CI/CD pipeline** integration with a pass/fail scoring system for automated security gates

Create the project directory and virtual environment

The security analyzer needs a dedicated folder with subdirectories for scanner modules and sample files to test against.

- Open **Terminal** on you local machine
- Navigate to the Desktop using `cd`

- Create your project folder, move into it, and create the subfolders by running these commands:
 - `mkdir security-analyzer`
 - `cd security-analyzer`
 - `mkdir scanners samples`
- Verify folders were created using `dir`

```
C:\Users\russe\OneDrive\Desktop>mkdir security-analyzer
C:\Users\russe\OneDrive\Desktop>cd security-analyzer
C:\Users\russe\OneDrive\Desktop\security-analyzer>mkdir scanners samples
```

```
C:\Users\russe\OneDrive\Desktop\security-analyzer>dir
Volume in drive C is WINDOWS
Volume Serial Number is 7212-4675

Directory of C:\Users\russe\OneDrive\Desktop\security-analyzer

05/18/2026  11:23 AM    <DIR>          .
05/18/2026  11:23 AM    <DIR>          ..
05/18/2026  11:23 AM    <DIR>          samples
05/18/2026  11:23 AM    <DIR>          scanners
               0 File(s)                0 bytes
               4 Dir(s)  86,455,717,888 bytes free

C:\Users\russe\OneDrive\Desktop\security-analyzer>|
```

Create and activate your virtual environment by running these Powershell commands:

`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

`python -m venv venv venv\Scripts\activate`

Install required Python packages

The scanner needs two packages: `boto3` to communicate with Amazon Bedrock, and `python-hcl2` to parse Terraform configuration files.

- Install both packages by running this command:

`pip install boto3 python-hcl2`

```

PS C:\Users\russe> python -m venv venv
PS C:\Users\russe> venv\Scripts\activate
(venv) PS C:\Users\russe> pip install boto3 python-hcl2
Collecting boto3
  Downloading boto3-1.43.9-py3-none-any.whl.metadata (6.6 kB)
Collecting python-hcl2
  Downloading python_hcl2-8.1.2-py3-none-any.whl.metadata (7.2 kB)

```

- Verify both packages installed correctly by running this quick import test:

```
python -c "import boto3; import hcl2; print('Both packages imported successfully')"
```

```

(venv) PS C:\Users\russe> python -c "import boto3; import hcl2; print('Both packages imported successfully')"
Both packages imported successfully
(venv) PS C:\Users\russe> |

```

Enable Claude 3.5 Haiku model access in Bedrock

Your analyzer will use Claude 3.5 Haiku on Amazon Bedrock to explain security risks in plain English

- Open the **AWS Management Console** at <https://console.aws.amazon.com> and sign in.
 - Make sure your region is set to **US East (N. Virginia)** (`us-east-1`) using the region selector in the top right corner.
 - Search for **Bedrock** in the top search bar and select **Amazon Bedrock**.
 - In the left navigation pane, under **Bedrock Discover**, choose **Model Catalog**.
-
- Find and select **Claude 4.5 Haiku** in the model list.
 - Choose **Next**.
 - If prompted with a use case details form for Anthropic, choose **Submit use case details**.

Amazon Bedrock > Model catalog > Claude Haiku 4.5

Amazon Bedrock

- Discover
 - Overview
 - Quickstart [New](#)
 - Model catalog
 - API keys
- Labs
 - Quotas [New](#)
 - Compare models [New](#)
- Test
 - Playground [New](#)
 - Watermark detection
 - Tokenizer
- Infer



Claude Haiku 4.5
By: Anthropic |

Claude Haiku 4.5 delivers near-frontier performance for a wide range of use cases, and stands out as one of the best coding and agent models—with the right speed and cost to power free products and high-volume user experiences.

Anthropic requires first-time customers to submit use case details before invoking a model, once per account or once at the organization's management account. The information you submit will be shared with Anthropic.

[Submit use case details](#)

Overview

Claude Haiku 4.5 delivers near-frontier performance for a wide range of use cases, and stands out as one of the best coding and agent models—with the right speed and cost to power free products and high-volume user experiences. Use cases: Powering free tier user experiences: Claude Haiku 4.5 delivers near-frontier performance at a cost and speed that makes powering free agent products and agentic use cases economically viable at scale. Real-time experiences: Claude

On this page

- [Overview](#)
- [Pricing](#)
- [Usage](#)

- Fill out the form with a brief description of your use case (e.g., security analysis of infrastructure code) and your website URL.
- Choose **Submit form**.
- Review the access changes, then choose **Submit**.
- Access is granted immediately after form submission

Amazon Bedrock > Model catalog > Claude Haiku 4.5

Amazon Bedrock

- Discover
 - Overview
 - Quickstart [New](#)
 - Model catalog
 - API keys



Claude Haiku 4.5
By: Anthropic |

Claude Haiku 4.5 delivers near-frontier performance for a wide range of use cases, and stands out as one of the best coding and agent models—with the right speed and cost to power free products and high-volume user experiences.

[Open in playground](#)

Create Intentionally insecure Sample Files

Create an insecure Terraform file

Terraform files (.tf) define cloud infrastructure as code. When written carelessly, they can expose entire environments to the internet.

- In VS Code, create a new file called `insecure_s3.tf` inside your `samples/` folder.
- Paste the following code into the file:

```
resource "aws_s3_bucket" "data_dump" {

  bucket = "company-data-dump-public"

  acl    = "public-read"

  tags = {
```

```
    Name      = "Data Dump Bucket"
    Environment = "production"
  }
}
```

```
resource "aws_security_group" "wide_open" {
  name      = "allow_all_traffic"
  description = "Extremely permissive security group"
  vpc_id    = "vpc-12345678"
```

```
  ingress {
    from_port = 0
    to_port   = 65535
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

```
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
resource "aws_db_instance" "production_db" {
```

```
allocated_storage = 20

engine = "mysql"

engine_version = "8.0"

instance_class = "db.t3.micro"

db_name = "productiondb"

username = "admin"

password = "supersecretpassword123"

storage_encrypted = false

publicly_accessible = true

skip_final_snapshot = true
}
```

💡 What security issues are hiding in this file?

This file defines three AWS resources, each with a different category of vulnerability:

- `aws_s3_bucket.data_dump` uses `acl = "public-read"`, meaning anyone on the internet can read its contents.
- `aws_security_group.wide_open` opens all TCP ports (0-65535) to `0.0.0.0/0`. This allows any IP address to connect to any service.
- `aws_db_instance.production_db` has `storage_encrypted = false` and `publicly_accessible = true`. An unencrypted, public-facing database is a critical risk.

Create an insecure IAM policy file

IAM policies control who can do what in your AWS account. Overly permissive policies are one of the most common causes of cloud breaches.

- Create a new file called `insecure_iam.json` inside your `samples/` folder.
- Paste the following JSON into the file:

```
{
```

```
"Version": "2012-10-17",  
"Statement": [  
  {  
    "Sid": "FullAdminAccess",  
    "Effect": "Allow",  
    "Action": "*",  
    "Resource": "*"   
  },  
  {  
    "Sid": "S3FullAccess",  
    "Effect": "Allow",  
    "Action": "s3:*",  
    "Resource": "*"   
  },  
  {  
    "Sid": "EC2FullAccess",  
    "Effect": "Allow",  
    "Action": "ec2:*",  
    "Resource": "*"   
  }  
]  
}
```

💡 What makes this policy dangerous?

Each statement in this policy violates the principle of least privilege:

- `FullAdminAccess` grants every possible action (`Action: "*"`) on every resource (`Resource: "*"`). This is equivalent to full administrator access.
- `S3FullAccess` and `EC2FullAccess` use service-level wildcards (`s3:*`, `ec2:*`) with no resource restrictions.
- None of the statements include a `Condition` block. There is no MFA requirement, IP restriction, or time constraint.

- Save both files in vscode
- Confirm both files exist by running this in your terminal: `cd samples dir`
- Your test data is ready. Next up, you will build the Terraform scanner that parses these `.tf` files and detects the vulnerabilities you just planted.

Create the scanner file with HCL2 parsing

For Python to treat the `scanners/` folder as an importable package, it needs an `__init__.py` file. This file can be empty.

- Create the package initializer file by running this powershell command from your `security-analyzer` folder:

```
New-Item -Path "__init__.py" -ItemType File -Force
```

```
dir scanners/
```

You should see `__init__.py` in the output.

Now you'll create the main scanner file. The `scan_terraform_file` function is the entry point. It opens a `.tf` file, uses `hcl2.load()` to parse the HCL2 syntax into a nested Python dictionary, then loops through each resource block to check for issues.

- Create a new file called `terraform_scanner.py` inside the `scanners/` folder.
- Add the following code to `scanners/terraform_scanner.py`

```
import json

import hcl2
```



```
def scan_terraform_file(file_path):

    findings = []

    with open(file_path, "r") as f:

        try:

            tf_config = hcl2.load(f)

        except Exception as e:

            print(f"Error parsing {file_path}: {e}")

            return findings

    resources = tf_config.get("resource", [])

    for resource_block in resources:

        for resource_type, instances in resource_block.items():

            for resource_name, config in instances.items():

                findings.extend(

                    check_resource(resource_type, resource_name, config)

                )

    return findings
```

- Save `scanners/terraform_scanner.py`.
- Verify the parsing works by running this quick import test in your terminal:

```
python -c "from scanners.terraform_scanner import scan_terraform_file; print('Import successful')"
```

```
(venv) PS C:\Users\russe\OneDrive\Desktop\security-analyzer> python -c "from scanners.terraform_scanner import scan_terraform_file; print('Import successful')"  
Import successful  
(venv) PS C:\Users\russe\OneDrive\Desktop\security-analyzer> |
```

Now add the `check_resource` function. This is a dispatcher that routes each resource to the correct detection function based on its type.

- Add the following function below `scan_terraform_file` in `scanners/terraform_scanner.py`:

```
def check_resource(resource_type, resource_name, config):  
  
    """Check a single resource for security issues."""  
  
    findings = []  
  
    if resource_type == "aws_s3_bucket":  
  
        findings.extend(check_s3_bucket(resource_name, config))  
  
    if resource_type == "aws_security_group":  
  
        findings.extend(check_security_group(resource_name, config))  
  
    if resource_type == "aws_db_instance":  
  
        findings.extend(check_rds_instance(resource_name, config))  
  
    return findings
```

Implement detection rules for common security issues

Each detection function looks at a resource's configuration dictionary and checks for a specific anti-pattern. When it finds one, it appends a finding dictionary with three keys: `resource` (which resource has the issue), `issue` (what the problem is), and `snippet` (the offending configuration).

- Add the S3 bucket detection function below `check_resource` in `scanners/terraform_scanner.py`:

```
def check_s3_bucket(resource_name, config):  
  
    """Check S3 bucket for public access and missing encryption."""  
  
    findings = []  
  
    acl = config.get("acl", "")  
  
    if acl in ["public-read", "public-read-write"]:  
  
        findings.append({  
  
            "resource": f"aws_s3_bucket.{resource_name}",  
  
            "issue": f"S3 bucket has public ACL: {acl}",  
  
            "snippet": json.dumps({"acl": acl}, indent=2),  
  
        })  
  
    return findings
```

Add the security group detection function below `check_s3_bucket`:

```
def check_security_group(resource_name, config):  
  
    """Check security group for overly permissive ingress rules."""  
  
    findings = []  
  
    ingress_rules = config.get("ingress", [])
```

```

if isinstance(ingress_rules, dict):
    ingress_rules = [ingress_rules]

for rule in ingress_rules:
    cidr_blocks = rule.get("cidr_blocks", [])
    from_port = rule.get("from_port", 0)
    to_port = rule.get("to_port", 0)

    if "0.0.0.0/0" in cidr_blocks:
        port_range = to_port - from_port
        if port_range > 100 or (from_port == 0 and to_port == 65535):
            findings.append({
                "resource": f"aws_security_group.{resource_name}",
                "issue": (
                    f"Security group allows wide port range "
                    f"({from_port}-{to_port}) open to the internet (0.0.0.0/0)"
                ),
                "snippet": json.dumps(rule, indent=2),
            })

return findings

```

Add the RDS instance detection function below `check_security_group`:

```

def check_rds_instance(resource_name, config):
    """Check RDS instance for missing encryption."""

    findings = []

```

```

storage_encrypted = config.get("storage_encrypted", False)

if not storage_encrypted:

    findings.append({

        "resource": f"aws_db_instance.{resource_name}",

        "issue": "RDS instance does not have storage encryption enabled",

        "snippet": json.dumps(

            {"storage_encrypted": storage_encrypted}, indent=2

        ),

    })

return findings

```

Save scanners/terraform_scanner.py

Test the scanner and verify the structured findings

Your scanner is complete. Time to run it against the insecure Terraform file you created earlier and confirm it detects all three planted vulnerabilities.

- Test the scanner by running this command from your **security-analyzer** folder:

```

python -c "

from scanners.terraform_scanner import scan_terraform_file

import json

findings = scan_terraform_file('samples/insecure_s3.tf')

print(f'Found {len(findings)} issues:')

for f in findings:

    print(json.dumps(f, indent=2))

"

```

```
(.venv) PS C:\Users\russe\OneDrive\Desktop\security-analyzer> python -c "
>> from scanners.terraform_scanner import scan_terraform_file
>> import json
>> findings = scan_terraform_file('samples/insecure_s3.tf')
>> print(f'Found {len(findings)} issues:') ...
>> "
Found 3 issues:
{
  "resource": "aws_s3_bucket.data_dump",
  "issue": "S3 bucket has public ACL: public-read",
  "snippet": "{\n  \"acl\": \"public-read\"\n}"
}
{
  "resource": "aws_security_group.wide_open",
  "issue": "Security group allows wide port range (0-65535) open to the internet (0.0.0.0/0)",
  "snippet": "{\n  \"from_port\": 0,\n  \"to_port\": 65535,\n  \"protocol\": \"tcp\",\n  \"cidr_blocks\": [\n    \"0.0.0.0/0\"\n  ]\n}"
}
{
  "resource": "aws_db_instance.production_db",
  "issue": "RDS instance does not have storage encryption enabled",
  "snippet": "{\n  \"storage_encrypted\": false\n}"
}
```

Create the IAM policy scanner file

IAM policies are JSON documents with a **Statement** array. Each statement specifies an Effect (Allow or Deny), Actions, and Resources. Your scanner needs to load this JSON and check each statement individually.

- Create a new file called `iam_scanner.py` inside the `scanners/` folder.
- Add the following code to create the main scanning function

```
import json
```

```
def scan_iam_policy(file_path):
```

```
    """Load and scan an IAM policy JSON file for security issues."""
```

```
    findings = []
```

```
    with open(file_path, "r") as f:
```

```
        try:
```

```
            policy = json.load(f)
```

```
        except json.JSONDecodeError as e:
```

```
            print(f"Error parsing {file_path}: {e}")
```

```
        return findings
```

```

# Extract the list of policy statements

statements = policy.get("Statement", [])

# Check each statement for security issues

for i, statement in enumerate(statements):

    findings.extend(check_statement(i, statement))

return findings

```

💡 What does this code do?

`json.load(f)` reads the IAM policy file and converts it into a Python dictionary. If the file isn't valid JSON, the scanner prints an error and returns an empty list instead of crashing.

The function iterates through each statement using `enumerate()` so it can track the statement index. This makes findings easier to locate when reviewing the report later.

Implement detection rules for policy statements

The most dangerous IAM patterns fall into three categories: wildcard actions that grant unrestricted access, wildcard resources that apply permissions to everything, and broad permissions without conditions like MFA requirements.

- In `scanners/iam_scanner.py`, add the `check_statement` function below the `scan_iam_policy` function:

```

def check_statement(index, statement):

    """Check a single IAM policy statement for security issues."""

    findings = []

    effect = statement.get("Effect", "")

```

```
# Only check Allow statements - Deny statements restrict access
```

```
if effect != "Allow":
```

```
    return findings
```

```
# Normalize actions and resources to lists
```

```
actions = statement.get("Action", [])
```

```
if isinstance(actions, str):
```

```
    actions = [actions]
```

```
resources = statement.get("Resource", [])
```

```
if isinstance(resources, str):
```

```
    resources = [resources]
```

```
# Check for wildcard actions
```

```
for action in actions:
```

```
    if action == "*":
```

```
        findings.append({
```

```
            "resource": f"Statement[{index}]",
```

```
            "issue": "IAM policy grants ALL actions (Action: *)",
```

```
            "snippet": json.dumps(statement, indent=2),
```

```
        })
```

```
        break
```

```
    elif action.endswith(".*"):
```

```
        findings.append({
```



```

        "resource": f"Statement[{index}]",
        "issue": (
            f"IAM policy grants all actions for a service: {action}"
        ),
        "snippet": json.dumps(statement, indent=2),
    })

```

Check for wildcard resources

if "*" in resources:

```

    findings.append({
        "resource": f"Statement[{index}]",
        "issue": "IAM policy applies to ALL resources (Resource: *)",
        "snippet": json.dumps(statement, indent=2),
    })

```

Check for missing conditions on sensitive permissions

condition = statement.get("Condition")

has_wildcard_action = "*" in actions or any(

```

    a.endswith(".*") for a in actions

```

)

if has_wildcard_action and not condition:

```

    findings.append({
        "resource": f"Statement[{index}]",
        "issue": (

```

```

        "Broad permissions granted without any Condition block "
        "(no MFA requirement, IP restriction, or time constraint)"
    ),
    "snippet": json.dumps(statement, indent=2),
})

return findings

```

Verify Scanner output

Test the scanner against your sample IAM policy by running this command from the **security-analyzer** folder:

```
python -c "from scanners.iam_scanner import scan_iam_policy; import json; findings = scan_iam_policy('samples/insecure_iam.json'); print(f'Found {len(findings)} issues'); print(json.dumps(findings[:2], indent=2))"
```

- You should see output showing multiple findings. The scanner should detect at least 7 issues across the three statements in your sample file (wildcard actions, wildcard resources, and missing conditions for each

```

(.venv) PS C:\Users\russe\OneDrive\Desktop\security-analyzer> python -c "from scanners.iam_scanner import scan_iam_policy; import json; findings = scan_iam_policy('samples/insecure_iam.json'); print(f'Found {len(findings)} issues'); print(json.dumps(findings[:2], indent=2))"
Found 9 issues
[
  {
    "resource": "Statement[0]",
    "issue": "IAM policy grants ALL actions (Action: *)",
    "snippet": "{\n  \"Sid\": \"FullAdminAccess\", \n  \"Effect\": \"Allow\", \n  \"Action\": \"*\", \n  \"Resource\": \"*\"
\n}"
  },
  {
    "resource": "Statement[0]",
    "issue": "IAM policy applies to ALL resources (Resource: *)",
    "snippet": "{\n  \"Sid\": \"FullAdminAccess\", \n  \"Effect\": \"Allow\", \n  \"Action\": \"*\", \n  \"Resource\": \"*\"
\n}"
  }
]

```

Integrate AWS Bedrock for AI analysis

Create the Bedrock analyzer file and system prompt

The system prompt is what shapes how Claude responds.

- Create a new file called `bedrock_analyzer.py` in the root of your `security-analyzer` folder (next to your `scanners/` directory).
- Add the imports and system prompt constant by pasting this code

```
import boto3
```

```
import json
```

```
from botocore.exceptions import ClientError
```

```
SYSTEM_PROMPT = """You are a senior cloud security engineer performing a security audit.
```

```
You will receive a list of security findings from automated scans of Terraform files and IAM policies.
```

```
For each finding, provide:
```

1. SEVERITY: Rate as CRITICAL, HIGH, MEDIUM, or LOW
2. EXPLANATION: A clear, plain-English explanation of why this is a security risk
3. ATTACK SCENARIO: A brief description of how an attacker could exploit this
4. REMEDIATION: Specific code or configuration changes to fix the issue

```
Format your response as a JSON array where each item has these fields:
```

- severity (string)
- explanation (string)
- attack_scenario (string)
- remediation (string)

```
Respond ONLY with the JSON array. No additional text before or after."""
```

Build the analyze_findings function with the Converse API

- The Converse API is AWS Bedrock's unified interface for sending messages to any supported model. It works the same way regardless of whether you use Claude, Llama, or another model behind Bedrock

```

def analyze_findings(findings, region="us-east-1"):

    """Send findings to Bedrock Claude for AI-powered analysis."""

    client = boto3.client("bedrock-runtime", region_name=region)

    model_id = "us.anthropic.claude-haiku-4-5-20251001-v1:0"

    findings_text = json.dumps(findings, indent=2)

    user_message = (

        f"Analyze the following security findings and provide your "

        f"assessment:\n\n{findings_text}"

    )

    try:

        response = client.converse(

            modelId=model_id,

            messages=[

                {

                    "role": "user",

                    "content": [{"text": user_message}],

                }

            ],

            system=[{"text": SYSTEM_PROMPT}],

            inferenceConfig={

                "maxTokens": 4096,

```

```

        "temperature": 0.2,

        },

    )

    response_text =
response["output"]["message"]["content"][0]["text"]

    # Strip markdown code blocks if present

    cleaned_text = response_text.strip()

    if cleaned_text.startswith("`"):

        first_newline = cleaned_text.index("\n")

        cleaned_text = cleaned_text[first_newline + 1:]

    if cleaned_text.endswith("`"):

        cleaned_text = cleaned_text[:-3]

    cleaned_text = cleaned_text.strip()

    ai_analysis = json.loads(cleaned_text)

except ClientError as e:

    print(f"Error calling Bedrock: {e}")

    ai_analysis = []

except json.JSONDecodeError:

    print("Warning: Could not parse AI response as JSON.")

    ai_analysis = []

```

```
report = {"findings": []}

for i, finding in enumerate(findings):

    enriched = {

        "resource": finding.get("resource", "Unknown"),

        "issue": finding.get("issue", "Unknown"),

        "snippet": finding.get("snippet", ""),

    }

    if i < len(ai_analysis):

        ai = ai_analysis[i]

        enriched["severity"] = ai.get("severity", "UNKNOWN")

        enriched["ai_explanation"] = ai.get("explanation", "")

        enriched["attack_scenario"] = ai.get("attack_scenario", "")

        enriched["remediation"] = ai.get("remediation", "")

    else:

        enriched["severity"] = "UNKNOWN"

        enriched["ai_explanation"] = "AI analysis unavailable"

        enriched["attack_scenario"] = ""

        enriched["remediation"] = ""

    report["findings"].append(enriched)

return report
```

💡 What does this code do?

The imports bring in `boto3` (the AWS SDK for Python), `json` for parsing responses, and `ClientError` for handling AWS API failures gracefully.

The `SYSTEM_PROMPT` tells Claude exactly how to format its response. By requesting a JSON array with specific fields, you can parse the AI output programmatically instead of scraping unstructured text.

💡 Why request JSON output from the AI?

Your code needs to merge Claude's analysis back into the scanner findings. If the AI returns free-form paragraphs, parsing becomes fragile and error-prone. Requesting a strict JSON structure means you can reliably extract severity, explanation, and remediation for each finding.

Hint: May need to install boto3

[pip install boto3](#)

Generate CLI and generate first report

Create the CLI entry point with file-type detection

The CLI needs to know which scanner to use for each file. A simple helper function that checks the file extension handles this routing decision.

- Create a new file called `security_analyzer.py` in the root of your `security-analyzer` folder (next to `bedrock_analyzer.py`).
- Add the imports and file-type detection function by pasting this code:

```
import argparse

import json

import sys

from scanners.terraform_scanner import scan_terraform_file

from scanners.iam_scanner import scan_iam_policy
```

```
from bedrock_analyzer import analyze_findings
```

Add the report formatter and main function

The report formatter takes the enriched findings from Bedrock and prints them with color-coded severity levels. This makes it easy to spot critical issues at a glance.

- In `security_analyzer.py`, add the `print_report()` function below `detect_file_type()`

```
def detect_file_type(file_path):

    """Detect whether a file is Terraform or IAM policy JSON."""

    if file_path.endswith(".tf"):

        return "terraform"

    elif file_path.endswith(".json"):

        return "iam"

    else:

        return None


def print_report(report):

    """Print a formatted security report to the terminal."""

    print("\n" + "=" * 60)

    print("  AI-POWERED CLOUD SECURITY ANALYSIS REPORT")

    print("=" * 60)

    if not report["findings"]:
```



```
print("\n No security issues found. Your configuration looks
good!")

print("=" * 60)

return

print(f"\n Total findings: {len(report['findings'])}")

print("-" * 60)

for i, finding in enumerate(report["findings"], 1):

    severity = finding.get("severity", "UNKNOWN")

    severity_colors = {

        "CRITICAL": "\033[91m",

        "HIGH": "\033[93m",

        "MEDIUM": "\033[33m",

        "LOW": "\033[36m",

    }

    color = severity_colors.get(severity.upper(), "\033[0m")

    reset = "\033[0m"

    print(f"\n [{color}{severity.upper()}{reset}] Finding #{i}")

    print(f" Resource: {finding.get('resource', 'N/A')}")

    print(f" Issue: {finding.get('issue', 'N/A')}")

    print(f"\n AI Analysis:")
```

```

        print(f"    {finding.get('ai_explanation', 'No analysis
available')}")

    if finding.get("remediation"):

        print(f"\n    Remediation:")

        print(f"    {finding.get('remediation')}")

    print("-" * 60)

print("\n" + "=" * 60)

```

💡 What does this code do?

The imports pull in all the modules you built in previous steps.

`scan_terraform_file` and `scan_iam_policy` handle detection, while `analyze_findings` sends results to Bedrock.

The `detect_file_type()` function checks the file extension to decide which scanner to use. Files ending in `.tf` go to the Terraform scanner, `.json` files go to the IAM scanner, and anything else is skipped.

💡 What does this code do?

The `print_report()` function formats each finding with ANSI color codes. Critical findings appear in red, high in yellow, medium in orange, and low in cyan.

For each finding, it displays the severity tag, the affected resource, the issue description, the AI-generated explanation, and remediation suggestions.

- Now add the `main()` function below `print_report()`. This is the orchestrator that ties all your modules together:

```
def main():
```

```
parser = argparse.ArgumentParser(

    description="AI-Powered Cloud Security Analyzer"

)

parser.add_argument(

    "files",

    nargs="+",

    help="Terraform (.tf) or IAM policy (.json) files to scan",

)

parser.add_argument(

    "--region",

    default="us-east-1",

    help="AWS region for Bedrock (default: us-east-1)",

)

args = parser.parse_args()

all_findings = []

for file_path in args.files:

    file_type = detect_file_type(file_path)

    if file_type == "terraform":

        print(f"Scanning Terraform file: {file_path}")
```

```
        findings = scan_terraform_file(file_path)

        all_findings.extend(findings)

    elif file_type == "iam":

        print(f"Scanning IAM policy file: {file_path}")

        findings = scan_iam_policy(file_path)

        all_findings.extend(findings)

    else:

        print(f"Skipping unsupported file type: {file_path}")

if not all_findings:

    print("\nNo security issues found!")

    return

print(f"\nFound {len(all_findings)} potential security issue(s).")

print("Sending findings to Amazon Bedrock for AI analysis...\n")

report = analyze_findings(all_findings, region=args.region)

print_report(report)

if __name__ == "__main__":

    main()
```

Run the analyzer against your sample files

Everything is wired up. Time to scan both of your intentionally insecure sample files and see what the AI has to say about them.

- Make sure your terminal is in the `security-analyzer` folder and your virtual environment is active.
- Run the security analyzer against both sample files by running this command:

```
python security_analyzer.py samples/insecure_s3.tf samples/insecure_iam.json
```

- Wait a few seconds for Bedrock to respond. You should see a formatted report appear in your terminal with color-coded severity levels for each finding.

```
(.venv) PS C:\Users\russe\OneDrive\Desktop\security-analyzer> python security_analyzer.py samples/insecure_s3.tf samples/insecure_iam.json

=====
AI-POWERED CLOUD SECURITY ANALYSIS REPORT
=====

Total findings: 12
-----

[CRITICAL] Finding #1
Resource: aws_s3_bucket.data_dump
Issue: S3 bucket has public ACL: public-read

AI Analysis:
The S3 bucket is configured with public-read ACL, making all objects in the bucket readable by anyone on the internet. This exposes sensitive data to unauthorized access and potential data breaches.

Remediation:
Change the ACL to private and use bucket policies with specific principal restrictions. Update the Terraform configuration: `acl = "private"` and implement bucket policies that explicitly grant access only to authorized AWS principals or services.
-----

[CRITICAL] Finding #2
Resource: aws_security_group.wide_open
Issue: Security group allows wide port range (0-65535) open to the internet (0.0.0.0/0)
```

Add CI/CD Integration with pass/fail scoring

Define the severity scoring map

Every security finding has a severity level from the AI analysis. You need to translate those text labels into numeric values so you can sum them into a single risk score.

- In `security_analyzer.py`, add a scoring dictionary near the top of the file (after the imports):

```
# Numeric severity values for CI/CD scoring
SEVERITY_SCORES = {
    "CRITICAL": 10,
    "HIGH": 7,
    "MEDIUM": 4,
    "LOW": 1,
}
```

Add the scoring function and new CLI arguments

You need a function that calculates the total risk score from a report, plus two new CLI arguments to control the threshold and output format.

- Add this function below the `SEVERITY_SCORES` dictionary:

```
def calculate_risk_score(report):
    """Calculate total risk score from findings."""
    total = 0
    for finding in report["findings"]:
        severity = finding.get("severity", "UNKNOWN").upper()
        total += SEVERITY_SCORES.get(severity, 0)
    return total
```

In the `main()` function, find the existing `parser.add_argument` calls and add these two new arguments after the `--region` argument:

```
parser.add_argument(
    "--output",
    choices=["text", "json"],
    default="text",
    help="Output format: text or json (default: text)",
)
```

Add the pass/fail logic and JSON output

The final piece replaces the simple `print_report(report)` call with logic that checks the score against the threshold, outputs in the requested format, and exits with the appropriate code.

- In `main()`, find this line at the end of the function:

```
report = analyze_findings(all_findings, region=args.region)
print_report(report)
```

- Replace those two lines with the following scoring and output logic:

```
report = analyze_findings(all_findings, region=args.region)

# Calculate the total risk score

risk_score = calculate_risk_score(report)

# Output in the requested format

if args.output == "json":

    output = {

        "findings": report["findings"],

        "risk_score": risk_score,

        "threshold": args.threshold,

        "passed": risk_score <= args.threshold,

    }

    print(json.dumps(output, indent=2))

else:

    print_report(report)

    print(f"\n Risk Score: {risk_score} | Threshold:
{args.threshold}")

# Exit with appropriate code for CI/CD

if risk_score > args.threshold:
```

```

        if args.output == "text":

            print(f"  FAILED: Risk score {risk_score} exceeds threshold {args.threshold}")

            sys.exit(1)

        else:

            if args.output == "text":

                print(f"  PASSED: Risk score {risk_score} is within threshold {args.threshold}")

                sys.exit(0)

```

Test the pass/fail behavior

Your sample files contain multiple Critical and High findings. The total score will be well above 7, so the default threshold should trigger a failure. Try different thresholds to see the behavior change.

- Run the analyzer with the default threshold to see it fail:

`python security_analyzer.py samples/insecure_s3.tf samples/insecure_iam.json`

```

=====
Risk Score: 70 | Threshold: 7
FAILED: Risk score 70 exceeds threshold 7
(.venv) PS C:\Users\russe\OneDrive\Desktop\security-analyzer> 

```

- You should see the security report followed by a FAILED message with the risk score.
- Check the exit code by running:

Echo \$?

- Now try a very high threshold to make it pass:

`python security_analyzer.py samples/insecure_s3.tf --threshold 100`

- Check the exit code again with `echo $?`. You should now see 0 (pass/true).
- Test the JSON output format by running:


```
python security_analyzer.py samples/insecure_iam.json --output json --threshold 50
```

You should see a JSON object with findings, risk_score, threshold, and passed fields. This is the format a CI/CD pipeline would consume

```
"risk_score": 40,  
"threshold": 50,  
"passed": true  
}  
(.venv) PS C:\Users\russe\OneDrive\Desktop\security-analyzer>
```

In this project I:

- Built a **Python CLI tool** that scans Terraform and IAM policy files for real-world security anti-patterns like wildcard permissions, public access, and missing encryption.
- Integrated **Amazon Bedrock's Converse API** with Claude 3.5 Haiku to generate intelligent risk explanations, attack scenarios, and remediation code.
- Generated professional **security reports** that combine automated scanning with AI-powered analysis, complete with severity ratings and actionable fixes.
- Added CI/CD pipeline integration with a pass/fail scoring system for automated security gates